distant leaps to any area of the domain, whenever a better solution is (randomly) found.

And, of course, many more intermediate options are available, like doing a fixed or random number of swaps per transition, or moving a whole segment of the solution elsewhere.

Which one of these works best? That's a good question, which is hard to answer from a theoretical point: let's try them out on our K10 graph, as shown in figure 17.8, and see which one gives the best average result; we will perform 1000 simulations each running for the same number of steps and with the same values for k and α, and compare the average cost for the solutions. We'll also assume that each sequence starts with "`New Carthage`", so that we cut duplicate solutions by a factor 10. Results are summed up in table 1.

Of course, the domain this time is still huge, ~3.6 million permutations, but not so large that we can't afford brute-force (armed with some patience, it takes a while): therefore, we know that the best possible solution has cost `1625` (as shown in figure 17.8).

```
["New Carthage", "Syracuse", "Buffalo",
"Pittsburgh", "Harrisburg", "Opal City",
"Metropolis", "Gotham City", "Civic City",]
```

Table 17.1 Average cost for the best solution found by simulated annealing